

2. Knapsack Problem

2.1. Problem definition (Author: Kelvin Siu)

The 0-1 knapsack problem aims to determine for a set of items with individual values and weights and a knapsack of a specified capacity: what combination of items can be stored inside the knapsack while maximising the total value of the items. The mathematical notation for the 0-1 knapsack problem is given through the Integer Linear Programming model (Cacchiani et al., 2022):

$$\max \sum_{j=1}^n p_j x_j$$

Subject to:

$$\sum_{j=1}^n w_j x_j \leq c, \quad x_j \in \{0,1\} \quad (j = 1, \dots, n)$$

Where:

x_j takes the value 1 if and only if item j is selected

- p_j = value of item
- x_j = variable for including or not including item
- w_j = weight of item
- c = knapsack capacity
- n = number of items

The formula calculates whether it's optimal to include a specific item which is decided by the value the item adds to the knapsack compared to its weight and how it will affect capacity. This calculation is done for each item and is summed consecutively whilst the weight of the combination of items does not exceed the knapsack capacity.

Assumptions	Limitations
• Items can either be taken completely or not at all	• Items cannot be broken down into smaller pieces
• Each item has a weight and value	• There is only one copy per item
• Weight and value are fixed, positive values	• Items are independent of each other i.e. the choice to include an item does not depend on another item
• Capacity of the knapsack is fixed	

2.2. An initial or naive solution (Author: Kelvin Siu)

A brute force approach was used to create an initial solution to this problem, using recursion to check all possible combinations of items. By analysing the python function used in The Brute Force Approach (W3Schools, 2025) as a base, this brute force method was able to be implemented in java.

Method	Description
knapsackBruteForce()	<pre> public class knapsack { public double knapsackBruteForce(Item[] items, double capacity, int itemNumber, ArrayList<Item> storedItems) { if(itemNumber == 0 capacity == 0) { return 0; // base case for recursion in case the knapsack is full or there are no items in the array } Item currentItem = items[itemNumber-1]; // fetching the last item in the array of items double currentValue = currentItem.getValue(); // recording the item's details double currentWeight = currentItem.getWeight(); if(currentWeight > capacity) { return knapsackBruteForce(items, capacity, itemNumber-1, storedItems); // if the weight of the current item is too large } else { ArrayList<Item> includeList = new ArrayList<Item>(storedItems); // creating array lists as copies of the input array ArrayList<Item> excludeList = new ArrayList<Item>(storedItems); double includeItemValue = currentValue + knapsackBruteForce(items, capacity-currentWeight, itemNumber-1, includeList); double excludeItemValue = knapsackBruteForce(items, capacity, itemNumber-1, excludeList); // if item isn't considered if (includeItemValue > excludeItemValue) { // checking to see if the value of the knapsack is greater with or without the item includeList.add(currentItem); storedItems.clear(); // clearing the input array list so that new items can be added to the knapsack storedItems.addAll(includeList); return includeItemValue; } else { storedItems.clear(); storedItems.addAll(excludeList); return excludeItemValue; } } } } </pre> Method for implementing recursion. Using a class called 'item' that stores value and weight information, the method fetches the last item in the array and if criteria related to the knapsack capacity are fulfilled, the value of including and excluding the item is compared, which is calculated by calling the method to evaluate the next item. This process repeats for each item being considered until either the knapsack reaches its capacity or there are no more items left.

readItems()	<pre> public Item[] readItems(String filename) throws FileNotFoundException { // method that will read the test File readItemData= new File(filename); Scanner fileScan = new Scanner(readItemData); int lineCount = -1; // counting the number of items in the file - starting at -1 so that the line count while(fileScan.hasNextLine()){ fileScan.nextLine(); lineCount++; } fileScan.close(); fileScan = new Scanner(readItemData); // closing and reopening the file scanner so that the scanner Item[] items = new Item[lineCount]; // initialising the item array with the correct number of items String itemData = ""; int i=0; while(fileScan.hasNextLine()){ itemData = fileScan.nextLine(); if(itemData.contains(s:",") == false) { // if the line read by the file scanner does not contain continue; } String[] itemArr = itemData.split(regex:","); // splitting the line containing item information by Item item = new Item(itemArr[0], Double.parseDouble(itemArr[1]), Double.parseDouble(itemArr[2])); items[i] = item; // storing the new item into the item array i++; } fileScan.close(); return items; } </pre> Fetches the list of items and their information by reading the input test file using a file and scanner object. Information is stored in a string array and string to double conversion is used to create the item object.
readCapacity()	<pre> public double readCapacity(String filename) throws FileNotFoundException { // File readItemData= new File(filename); Scanner fileScan = new Scanner(readItemData); String itemData = ""; double capacity = 0; while(fileScan.hasNextLine()) { itemData = fileScan.nextLine(); if(itemData.contains(s:",") == false) { capacity = Double.parseDouble(itemData); // if the line read by break; } continue; } fileScan.close(); return capacity; } </pre> Fetches the capacity of the knapsack which is located on the last line of the file.
main()	<pre> public static void main(String[] args) throws Exception { Knapsack ks = new Knapsack(); // creating new knapsack object // calling readItems and readCapacity methods to store information about items and knapsack capacity. Uncomment lines to test different files Item[] items = ks.readItems(filename:"test1.txt"); double capacity = ks.readCapacity(filename:"test1.txt"); System.out.println("Test 1:\nknapsack capacity: " + capacity + "\nitems in the knapsack:"); /*Item[] items = ks.readItems("test2.txt"); double capacity = ks.readCapacity("test2.txt"); System.out.println("Test 2:\nknapsack capacity: " + capacity + "\nitems in the knapsack:");*/ /*Item[] items = ks.readItems("test3.txt"); double capacity = ks.readCapacity("test3.txt"); System.out.println("Test 3:\nknapsack capacity: " + capacity + "\nitems in the knapsack:");*/ /*Item[] items = ks.readItems("test4.txt"); double capacity = ks.readCapacity("test4.txt"); System.out.println("Test 4:\nknapsack capacity: " + capacity + "\nitems in the knapsack:");*/ /*Item[] items = ks.readItems("test5.txt"); double capacity = ks.readCapacity("test5.txt"); System.out.println("Test 5:\nknapsack capacity: " + capacity + "\nitems in the knapsack:");*/ ArrayList<Item> storedItems = new ArrayList<Item>(); double maxValue = ks.knapsackBruteForce(items, capacity, items.length, storedItems); // calling the method to process items to be stored in the knapsack double totalWeight = 0; for (int i=0; i < storedItems.size(); i++) { System.out.println(storedItems.get(i).getName() + " (Value: " + storedItems.get(i).getValue() + ", Weight: " + storedItems.get(i).getWeight() + ")"); totalWeight += storedItems.get(i).getWeight(); // tallying up the weight of each item in the knapsack } System.out.println("Number of items in the knapsack: " + storedItems.size()); // displaying knapsack information System.out.println("Total Weight: " + totalWeight); System.out.println("Maximum Value: " + maxValue + "\n"); } </pre>

	The main method starts by creating a new knapsack object. The methods for reading information from files are called, where the desired test file can be chosen by uncommenting the correct blocks of code. An array list of items is created and used as input to the brute force method. Using a for loop, the information of each item in the knapsack is printed and the weight of each item is tallied up. Finally, the information about the knapsack is printed as a list to be viewed. See appendix 2a for input test files and outputs.
--	--

2.6. Discussion of practical and real-world applications (Authors: Kelvin Siu)

According to Steven S. Skiena, the Knapsack Problem is the 18th most popular algorithm and 4th most needed implementation (1999, p.68-69). The applications include:

- A shipping company that needs to determine the most optimal method of loading cargo onto a vehicle (Sedgewick et al., 1998, p.224). Knapsack problem helps calculate how to get the most value out of the cargo put onto the vehicle while adhering to the vehicle's capacity
- Manufacturing organisations that need to cut raw materials into industry-standard sizes in a way that minimises the amount of material wasted (Kellerer et al., 2004, p.449). Applying knapsack aids in maximising the profit gained through the cuts of material
- Capital budgeting problems where knapsack is used to calculate the optimal selection of financial investments for projects around a fixed budgetary constraint, where the budget represents the knapsack capacity (Kellerer et al., 2004, p.461). Essential application for maximising company profits on investments.